# Heart Disease MLOps Platform

End-to-end ML model development, CI/CD and production deployment

| | |
|---|---|
| Course | Machine Learning Operations (MLOps), AIMLCZG523 |
| Assignment | Assignment 01 (50 marks) |
| Student | Bhavya Goyal  ID: 2025AE05816 |
| Dataset | UCI Heart Disease, repository id 45 |
| Code repository | https://github.com/bhavya-goyal-22/-mlops |
| Demo video | recordings/merged_recording.mkv (short walkthrough of the full pipeline) |

## Summary

For this assignment I built a small but complete MLOps pipeline around one question: can 13 routine clinical measurements tell us whether a patient has heart disease? I downloaded and cleaned the UCI data, explored it, tuned and compared two classifiers with MLflow, and exported the winner to ONNX. I then wrapped it in a FastAPI service, packaged that in Docker, ran two replicas on a local Minikube cluster, and watched it with Prometheus and Grafana. A GitHub Actions workflow lints, tests and retrains everything on each push, so the pipeline can be rebuilt from scratch.

**What I found.** The random forest (depth 3, 100 trees) has the best cross-validated ROC-AUC (**0.904**, against 0.893 for logistic regression) and is the packaged model. On a 60-patient holdout that was not used for selection, both models reach ROC-AUC of about 0.95.

## How the assignment tasks map to this report

| Assignment task | Marks | Where covered |
|---|---|---|
| 1. Data acquisition and EDA | 5 | Sections 3 and 4 |
| 2. Feature engineering and model development | 8 | Section 5 |
| 3. Experiment tracking (MLflow) | 5 | Section 6 |
| 4. Model packaging and reproducibility | 7 | Sections 2 and 7 |
| 5. CI/CD pipeline and automated testing | 8 | Section 8 |
| 6. Model containerization | 5 | Section 9 |
| 7. Production deployment (Kubernetes) | 7 | Section 10 |
| 8. Monitoring and logging | 3 | Section 11 |
| 9. Documentation and reporting | 2 | This report; architecture in Section 1 |

# 1. System overview and architecture

The model is a binary classifier: given 13 clinical measurements, predict whether heart disease is present. Training stays in scikit-learn. I deliberately do not serve the pickled scikit-learn object; the chosen pipeline is exported to ONNX and executed inside the API by ONNX Runtime, so the container does not depend on the MLflow store or the training environment.

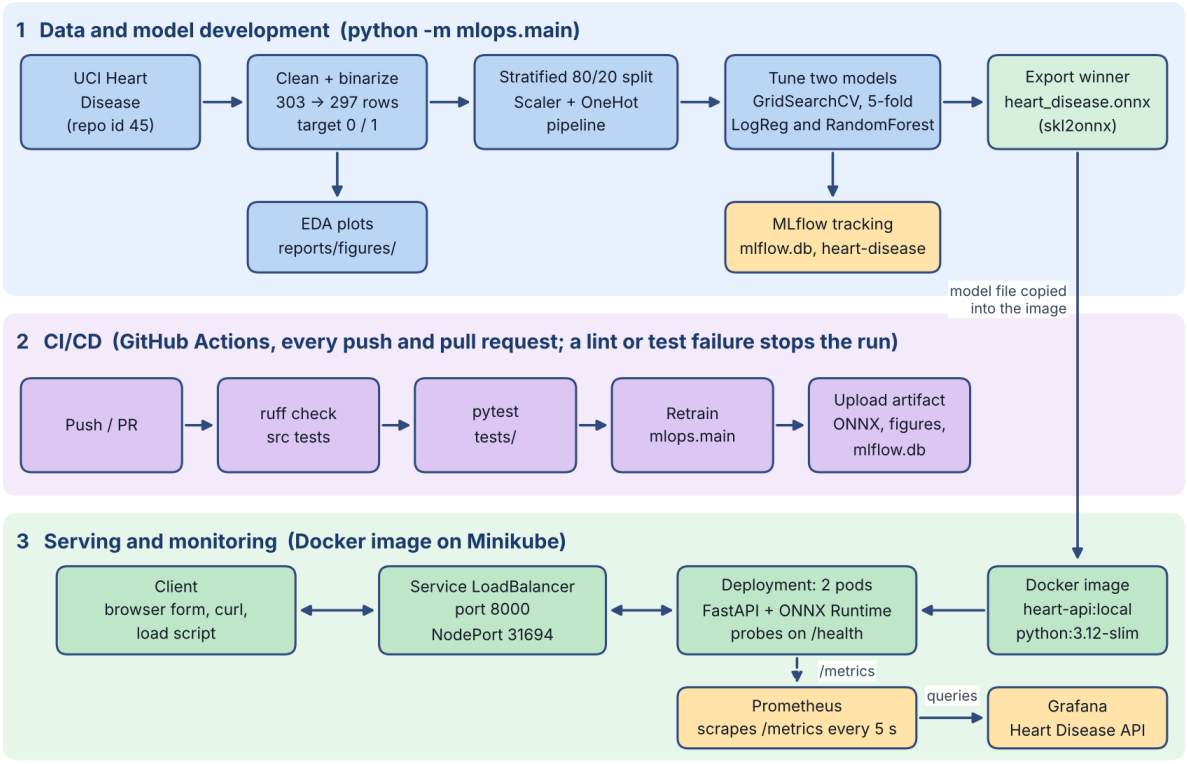



*Figure 1. Architecture: development, CI/CD and serving/monitoring.*

## Where each piece lives

| Piece | Location |
|---|---|
| Download, clean, EDA, training | src/mlops/, started by python -m mlops.main |
| Settings | configs/config.yaml |
| Cleaned tables and plots | data/cleaned/, reports/figures/ |
| Experiment log | MLflow experiment heart-disease, SQLite file mlflow.db |
| Packaged model | models/heart_disease.onnx |
| Tests / CI | tests/, .github/workflows/ci.yml |
| API image | Dockerfile, image name heart-api:local |
| Cluster | k8s/ manifests on Minikube (no Helm chart) |
| Metrics | Prometheus scrapes /metrics; Grafana dashboard **Heart Disease API** |

# 2. Setup and installation

**Prerequisites:** Python 3.12, Git, Docker, Minikube and kubectl (the last three only for the deployment steps). From the project root:

```
git clone https://github.com/bhavya-goyal-22/-mlops.git && cd -mlops
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

requirements.txt pins the stack used for both training and the container: pandas, scikit-learn, matplotlib, seaborn, MLflow, skl2onnx, onnxruntime, FastAPI, uvicorn, prometheus-client, pytest and ruff. Train, test and open the experiment UI:

```
PYTHONPATH=src python -m mlops.main
PYTHONPATH=src pytest tests/ -q
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Training writes data/cleaned/, reports/figures/, models/heart_disease.onnx and the MLflow store. The API can also run without Docker:

```
PYTHONPATH=src uvicorn mlops.api:app --host 0.0.0.0 --port 8000
```

Docker and the cluster are covered in Sections 9 and 10. A clean machine needs only the requirements file and the commands above; GitHub Actions repeats lint, test and train on its own runner (Section 8).

# 3. Data acquisition

The loader (src/mlops/data_loader.py) calls ucimlrepo.fetch_ucirepo(id=45) and saves data/raw/heart_disease.csv if it is not already present. The raw table has 303 rows and 14 columns: 13 inputs plus target. The original target is the angiographic status num, coded 0 to 4, not a binary label:

| Code | 0 (no disease) | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| Rows | 164 | 55 | 36 | 35 | 13 |

Missing values occur only in ca (4 rows) and thal (2 rows). Those six rows are dropped and both columns are stored as integers. The target is binarized: 0 stays "no disease" and any value above 0 becomes "disease" (Preprocessor.clean in src/mlops/preprocessing.py). After cleaning, data/cleaned/heart_disease.csv has **297 rows**:

| Class | Rows | Share |
|---|---|---|
| 0, no disease | 160 | 53.9% |
| 1, disease | 137 | 46.1% |

The classes are close enough that accuracy is usable, but ROC-AUC is the score used to choose the model so that a small shift in threshold does not decide the winner. I dropped the six rows (2%) rather than imputing them: the loss is small, and it avoids baking an imputation assumption into the model.

## Columns

| Column | Meaning | Column | Meaning |
|---|---|---|---|
| age | Age in years | exang | Exercise-induced angina (1/0) |
| sex | 1 male, 0 female | oldpeak | ST depression induced by exercise |
| cp | Chest-pain type | slope | Slope of peak exercise ST segment |

| Column | Meaning | Column | Meaning |
|---|---|---|---|
| trestbps | Resting blood pressure | ca | Major vessels coloured by fluoroscopy |
| chol | Serum cholesterol | thal | Thalassemia / defect code |
| fbs | Fasting blood sugar > 120 mg/dl | thalach | Maximum heart rate achieved |
| restecg | Resting electrocardiogram | target | 0 no disease, 1 disease |

Numeric summary of the cleaned table: age averages 54.5 years (range 29 to 77), resting blood pressure 131.7, cholesterol 247.4, maximum heart rate 149.6 and oldpeak 1.1.

# 4. Exploratory data analysis

src/mlops/eda.py writes the figures to reports/figures/ before the split, and the train-versus-test figures after it. The same images are copied under screenshots/ for the submission folder.

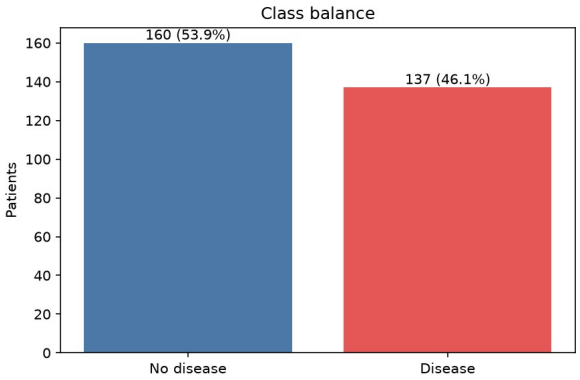



*Figure 2. Class balance, 297 cleaned rows.*

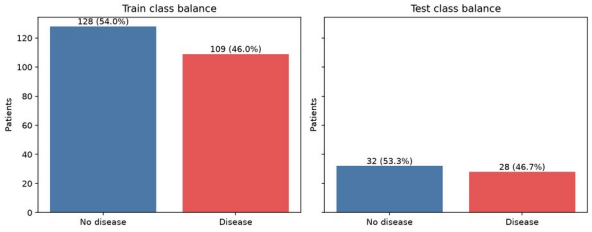



*Figure 3. Class balance in train and test.*

**Class balance.** The two classes are close, with a small majority of patients without disease (Figure 2). The stratified split keeps that ratio in both train and test (Figure 3).

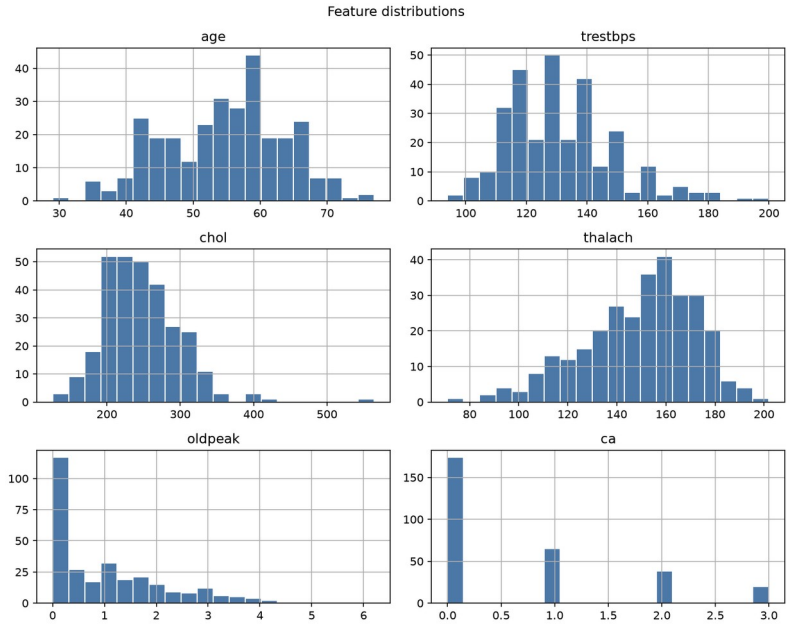



*Figure 4. Histograms of age, resting BP, cholesterol, maximum heart rate, oldpeak and ca.*

**Distributions.** Cholesterol and oldpeak are right-skewed, which supports scaling the numeric inputs. ca is a small integer and mostly 0.

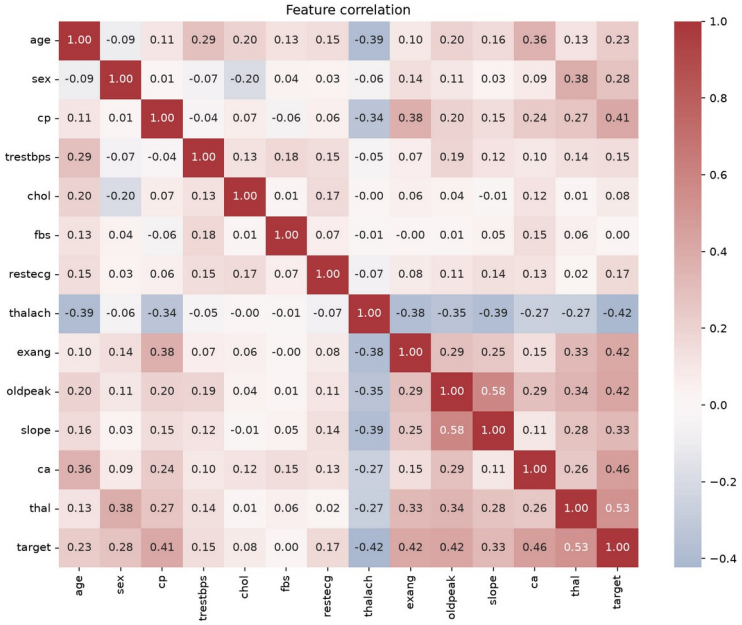



*Figure 5. Correlation of numeric columns with each other and with the target.*

**Correlations.** The strongest links with disease are thal (0.53), ca (0.46), oldpeak (0.42), exang (0.42) and cp (0.41). Maximum heart rate moves the other way (thalach -0.42): higher recorded peaks sit with the no-disease class. Fasting blood sugar is almost unrelated (0.00). These signs match the usual reading of this dataset and are why ca, thal, oldpeak and heart rate stay in the model.

# 5. Feature engineering and model development

The split is an 80/20 stratified split with random_state 42, so each side keeps the same disease rate. Saved files: data/cleaned/train.csv (237 rows) and data/cleaned/test.csv (60 rows).

| Split | Rows | No disease | Disease |
|---|---|---|---|
| Train | 237 | 128 (54.0%) | 109 (46.0%) |
| Test | 60 | 32 (53.3%) | 28 (46.7%) |

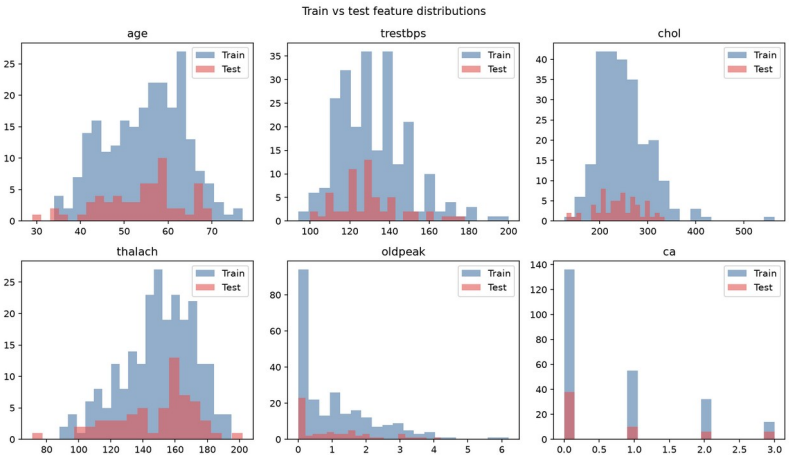



*Figure 6. Feature distributions, train versus test.*

The histograms on the two splits overlap. The test set is small, so a few bins differ, but there is no obvious shift that would make the holdout unusable.

## Preprocessing pipeline

Each candidate is a scikit-learn `Pipeline`, so exactly the same transformation is applied in training, cross-validation and serving:

1. **Numeric:** `StandardScaler` on age, trestbps, chol, thalach, oldpeak and ca.
2. **Categorical:** `OneHotEncoder(drop="first", handle_unknown="ignore")` on cp, restecg, slope and thal.
3. **Binary:** sex, fbs and exang passed through unchanged (already 0/1).

Scaling and encoding are fit only on the training folds inside the pipeline, so test rows never leak into the scaler.

## Models and tuning

Both models are tuned with `GridSearchCV`, 5-fold stratified cross-validation, scoring `roc_auc`:

| Model | Search grid |
|---|---|
| Logistic regression (`max_iter` 1000) | `C` in {0.1, 1.0, 10.0} |
| Random forest | `n_estimators` in {100, 200}, `max_depth` in {3, 5, 10} |

Logistic regression is the linear baseline: its coefficients are readable and it is hard to overfit on 237 rows unless `C` is large. Random forest is the non-linear comparison and can use interactions such as vessel count with ST depression. The search chooses depth and tree count instead of leaving the forest at the library default, which would memorize the training set.

After the search, the best pipeline of each model is scored again with `cross_validate` on accuracy, precision, recall and ROC-AUC. The holdout is scored once, after selection, and is never used to choose the winner. **Winner rule:** the higher cross-validated ROC-AUC is exported; on the finished run that is the random forest.

## Results

| 5-fold CV mean (train) | Logistic regression (C = 0.1) | Random forest (100 trees, depth 3) |
|---|---|---|
| Accuracy | 0.818 | 0.823 |
| Precision | 0.843 | 0.876 |
| Recall | 0.742 | 0.723 |
| ROC-AUC | 0.893 | **0.904** |

| Holdout, 60 patients | Logistic regression | Random forest |
|---|---|---|
| Accuracy | 0.850 | 0.817 |
| Precision | 0.880 | 0.870 |
| Recall | 0.786 | 0.714 |
| ROC-AUC | 0.953 | 0.952 |

The forest wins the selection metric by about 0.01 AUC. On the 60-row holdout the logistic model has higher accuracy and recall, and the two AUCs are almost identical (0.953 and 0.952). That gap is

expected: the holdout is small, and a depth-3 forest is slightly more conservative (higher precision, lower recall). The export follows the rule written in code, cross-validated ROC-AUC, so the API serves the random forest. Training-set scores for the forest (accuracy 0.882, ROC-AUC 0.954) sit above the CV scores, which is normal, and the depth cap keeps that gap modest.

**My reading.** ROC-AUC near 0.90 under CV and near 0.95 on this holdout is strong for 13 clinical fields. Recall around 0.72 means some disease cases are missed; for a screening use case this is the number I would improve first, for example with a lower decision threshold. This submission keeps the default 0.5 threshold.

# 6. Experiment tracking with MLflow

MLflow 3.16 is the experiment log. The tracking URI is the SQLite file `mlflow.db`; the UI is only a view of that file (`mlflow ui --backend-store-uri sqlite:///mlflow.db`). The experiment is `heart-disease`. One parent run, `compare-models`, contains a nested run per model (`logistic_regression`, `random_forest`). The finished parent used for the figures is run `e61b096834604696ac7b1f2ddd338645` (about 30 s, status Finished, source `main.py`), whose note records 237 train rows and 60 validation rows.

**What each run stores:**

- Parameters, including the chosen `C`, or `max_depth` and `n_estimators`.
- CV metrics, and holdout accuracy, precision, recall, ROC-AUC and average precision.
- Train and validation class counts, and system metrics (CPU, memory, disk).
- The fitted pipeline as an MLflow sklearn model.
- ROC, precision-recall and confusion-matrix figures for train and validation.
- Artifacts: raw and cleaned CSVs, the split, `config.yaml`, `requirements.txt` and the EDA figures; the ONNX file is logged on the parent run under `onnx/`.

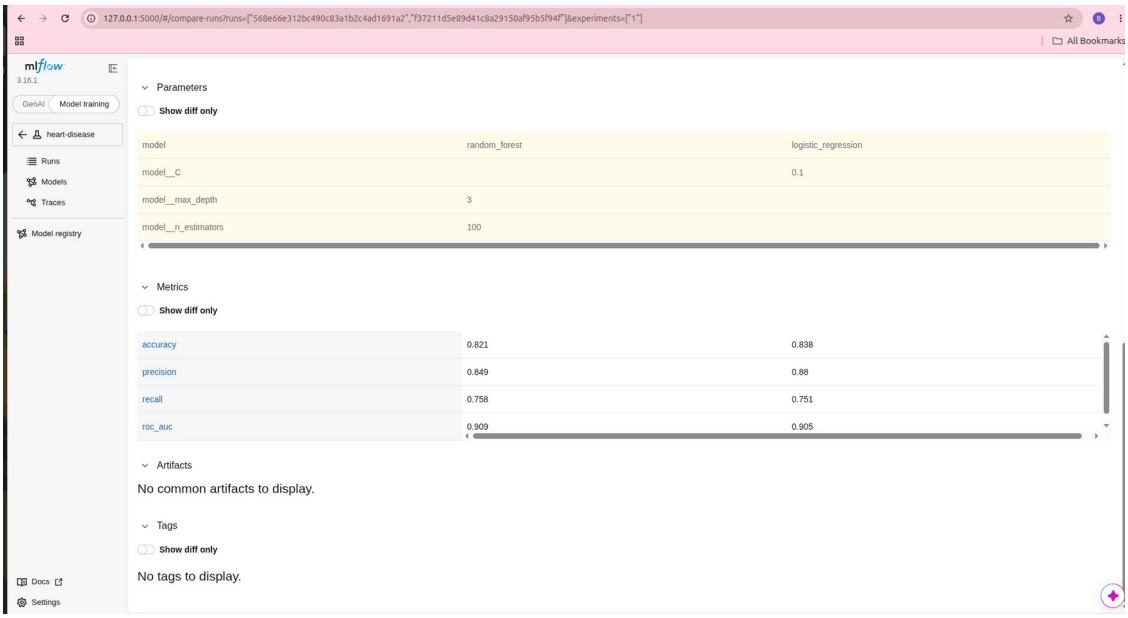



*Figure 7. MLflow compare page (earlier autolog run: RF ROC-AUC 0.909, LR 0.905).*

The comparison screenshot is an earlier side-by-side of the autolog metrics; its parameters match the final search (forest depth 3 and 100 trees, logistic `C` 0.1). The tables in Section 5 come from the later `cross_validate` log that the export uses.

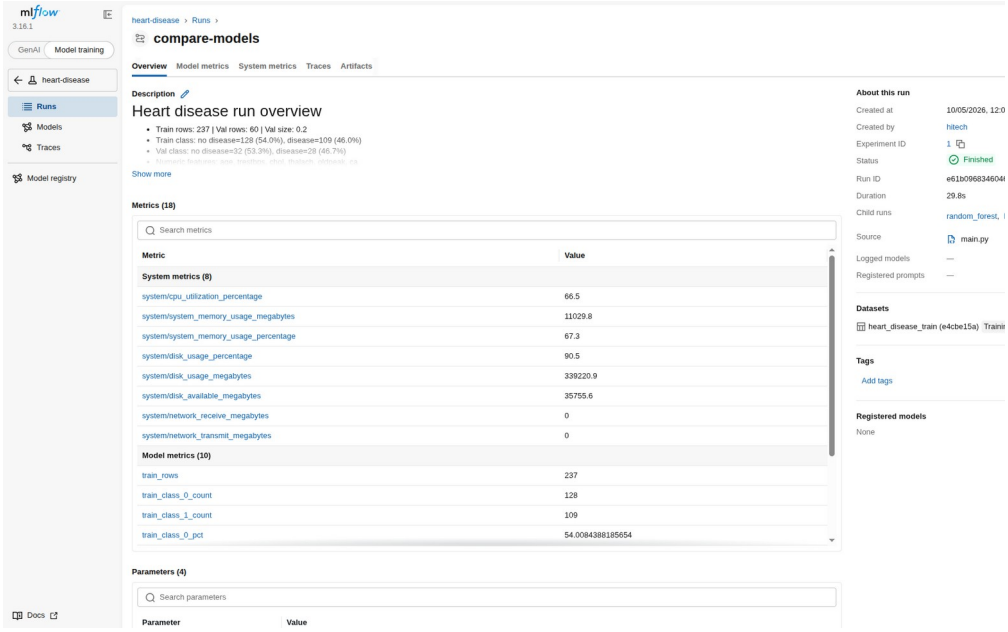



*Figure 8. Parent run overview: counts, system metrics and the logged dataset.*

# 7. Model packaging and reproducibility

skl2onnx.convert_sklearn converts the winning pipeline, scaler and one-hot encoder included, to ONNX. Inputs are one float tensor per column, which is how ONNX Runtime expects the 13 fields. The file is models/heart_disease.onnx. src/mlops/predict.py loads it, checks that all 13 names are present, runs the session and returns, for example:

```
{"prediction": 0, "probability": 0.336}
```

prediction is 0 or 1 and probability is the score for class 1. The sample row from the API form (age 63, sex 1, cp 1, trestbps 145, chol 233, fbs 1, restecg 2, thalach 150, exang 0, oldpeak 2.3, slope 3, ca 0, thal 6) scores as **no disease, probability 0.336**.

**Reproducibility.** The same result can be rebuilt from a clean checkout because (a) dependencies are pinned in requirements.txt, (b) the preprocessing is part of the saved pipeline rather than separate code, (c) the split and models use a fixed random_state, (d) all settings live in configs/config.yaml, and (e) MLflow stores the parameters, config and requirements next to each run. Serving never unpickles the scikit-learn object, which keeps the Docker image independent of the MLflow store.

# 8. Automated testing and CI/CD

Pytest covers four behaviours the pipeline must keep:

| File | What it checks |
|---|---|
| tests/test_data.py | Missing ca/thal rows are dropped, the target becomes 0/1, one-hot removes cp, and a CSV on disk loads |
| tests/test_model.py | Both model names exist, the preprocessor has numeric, categorical and binary steps, and each model can fit and predict 0/1 |
| tests/test_predict.py | The ONNX file returns a class and a probability in [0, 1] |
| tests/test_api.py | / has the title, /health is ok, /monitor and /metrics expose heart_api_requests_total, and POST /predict returns a class and probability |

GitHub Actions runs .github/workflows/ci.yml on every push and pull request on ubuntu-latest:

1. Checkout the repository.
2. Set up Python 3.12 and pip install -r requirements.txt.
3. ruff check src tests; the job fails if lint fails.
4. PYTHONPATH=src pytest tests/ -q.
5. PYTHONPATH=src python -m mlops.main, which retrains and rewrites the ONNX file.
6. Upload models/heart_disease.onnx, reports/figures and mlflow.db as artifact pipeline-artifacts, also when an earlier step fails (if: always()).

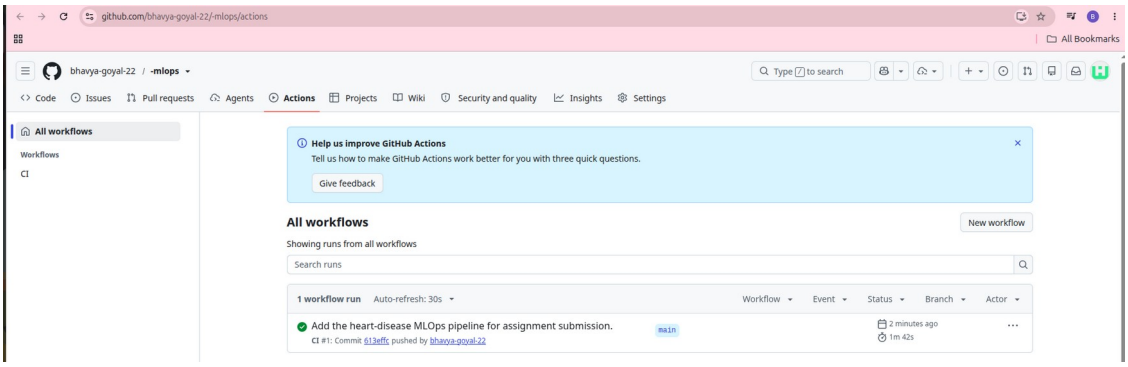



*Figure 9. GitHub Actions: CI run #1 on main finished successfully in 1m 42s.*

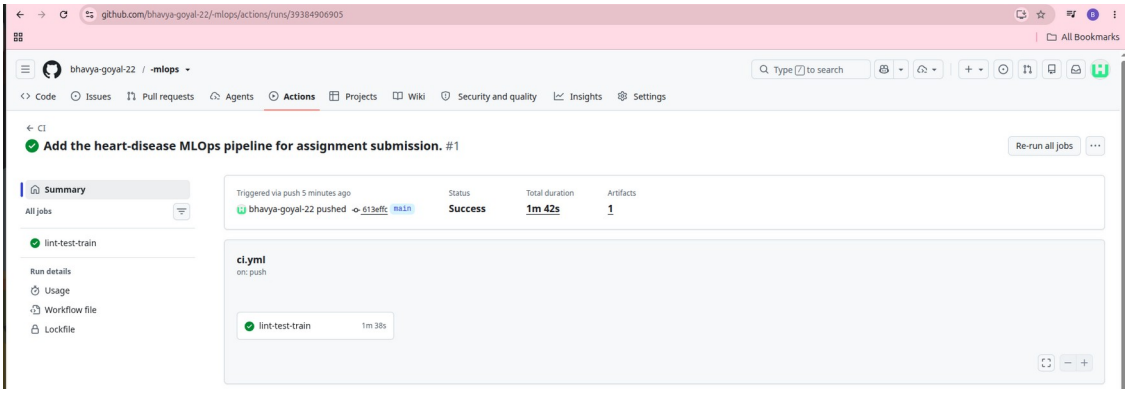



*Figure 10. Run summary: job lint-test-train passed, status Success, 1 artifact (pipeline-artifacts) produced.*

# 9. API and containerization

FastAPI is the application and uvicorn listens on port 8000. A Pydantic model validates the 13 fields before they reach the predictor (sex, fbs and exang must be 0 or 1).

| Method | Path | Behaviour |
|---|---|---|
| GET | / | HTML form |
| GET | /predict | Lists the 13 feature names and says the method is POST |
| POST | /predict | JSON in, prediction and probability out |
| GET | /health | {"status": "ok", "model_path": ...} |
| GET | /monitor | Last requests: method, path, status, latency (ms) |
| GET | /metrics | Prometheus text |

| Method | Path | Behaviour |
|---|---|---|
| GET | /docs | OpenAPI page |

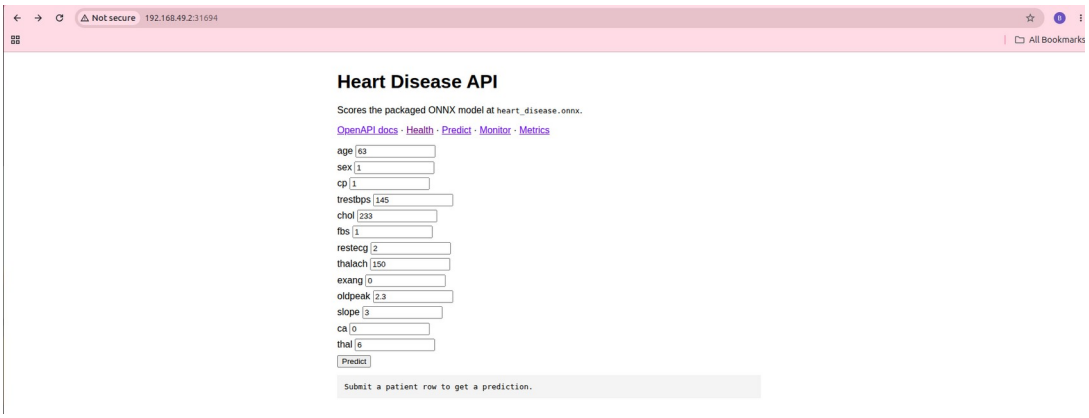



*Figure 11. The form page; submitting it posts to /predict and prints the JSON.*

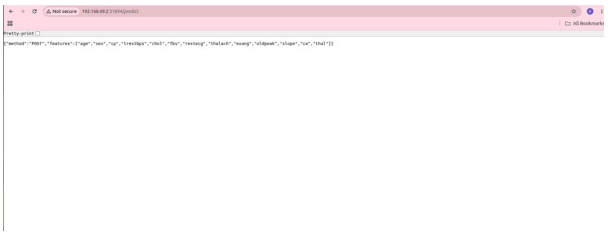


*Figure 12. GET /predict (contract).*

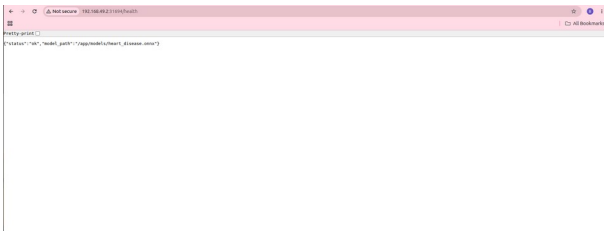


*Figure 13. GET /health.*

Figure 12 shows the contract at http://192.168.49.2:31694/predict, not a scored patient; a scored patient is the POST described in Section 7.

## Docker image

The image heart-api:local is built from Dockerfile: base python:3.12-slim; pip install -r requirements.txt; copy src/, configs/ and models/heart_disease.onnx; start uvicorn mlops.api:app --host 0.0.0.0 --port 8000.

```
docker build -t heart-api:local .
docker run --rm -p 8000:8000 heart-api:local

curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
      "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
```

Expected response: {"prediction": 0, "probability": 0.336}. The image is the packed filesystem and the container is a running copy of it; Kubernetes never reads the laptop's src/ folder.

**[INSERT SCREENSHOT: docker build and docker run output, plus the curl POST /predict response]**

*Figure 14. Container build, run and a scored prediction.*

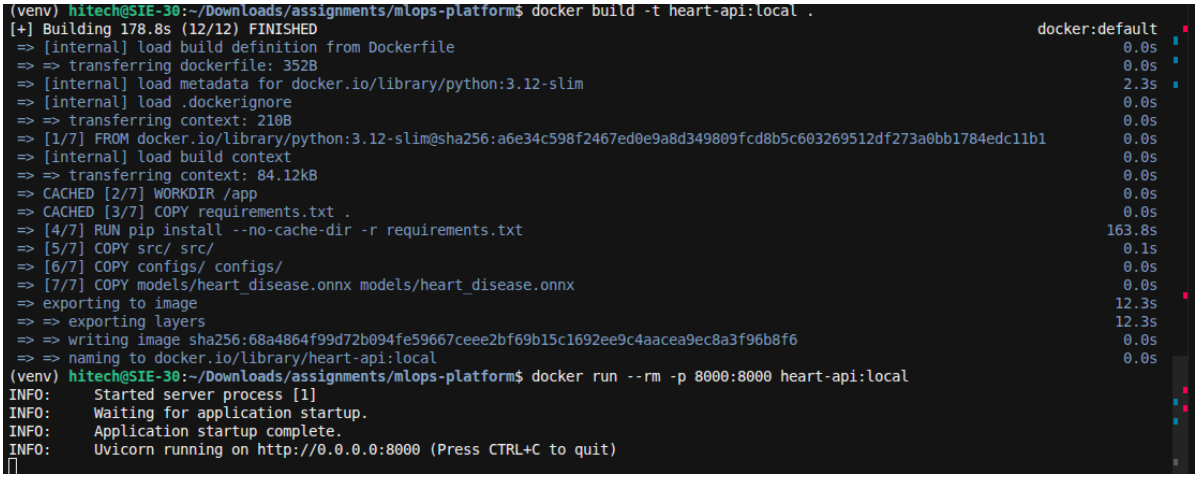



*Figure 14. Successful Docker image build and local container startup; Uvicorn reports the API running on port 8000.*

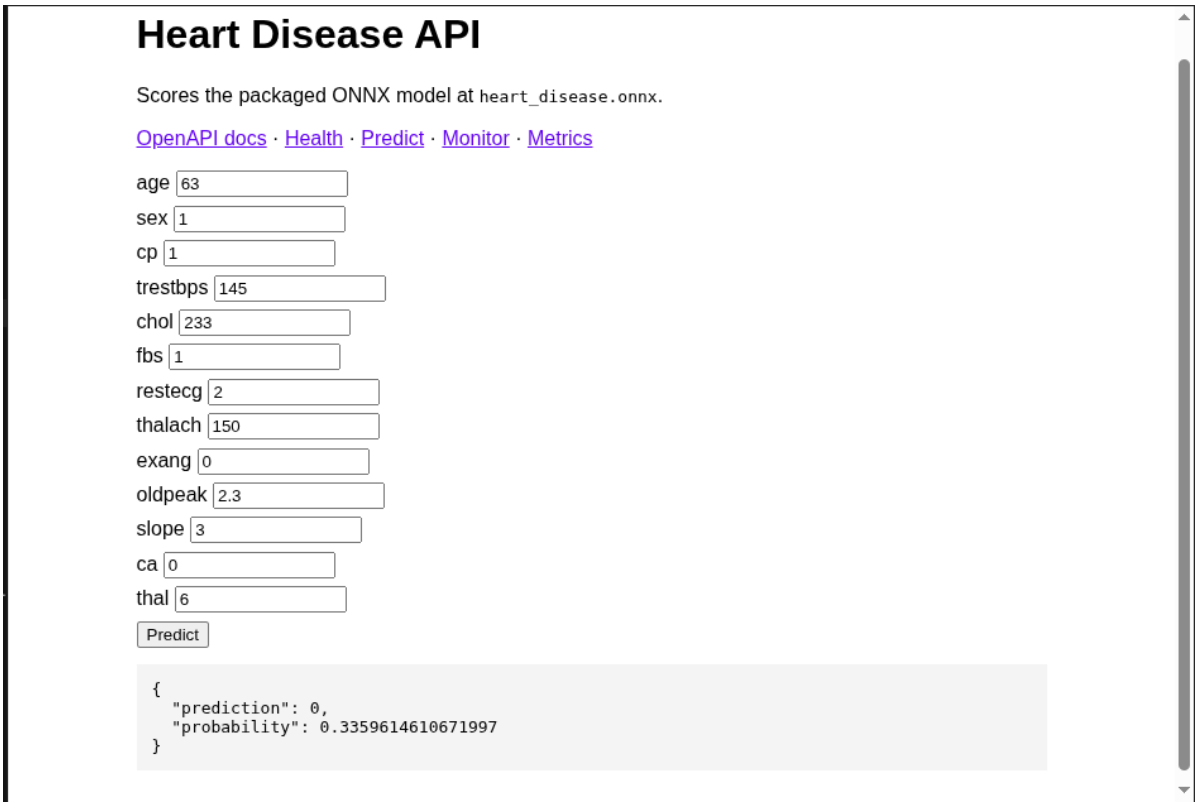



*Figure 15. Successful sample prediction from the running Heart Disease API: prediction 0, with class-1 probability approximately 0.336.*

# 10. Production deployment on Kubernetes

Production deployment for this assignment is local Kubernetes with Minikube (Docker driver). The brief allows a deployment manifest or a Helm chart; k8s/ is the manifest set.

```
minikube start --driver=docker
minikube image load heart-api:local
kubectl apply -f k8s/
kubectl get pods,svc
minikube service heart-api --url
```

minikube image load is required because Minikube has its own Docker daemon, and imagePullPolicy: Never tells the pods to use the loaded image instead of pulling one.

- **Deployment** (k8s/deployment.yaml): two replicas of heart-api:local on port 8000, so one pod can restart while the other serves. Readiness and liveness probes are HTTP GET /health; a failing liveness probe restarts the pod.
- **Service** (k8s/service.yaml): type LoadBalancer on port 8000, a stable address that spreads traffic across the pods. On Minikube the external IP stays <pending> without minikube tunnel, but the NodePort works (31694).

| Service | Port | NodePort used in browser | Pods (ready, status, IP) |
|---|---|---|---|
| heart-api | 8000 | 31694 | 2 pods, 1/1 Running, 10.244.0.7 and 10.244.0.8 |
| prometheus | 9090 | 30275 | 1/1 Running, 10.244.0.6 |
| grafana | 3000 | 31957 | 1/1 Running, 10.244.0.5 |

```
NAME                              READY   STATUS    RESTARTS   AGE     IP           NODE       NOMINATED NODE   READINESS GATES
pod/grafana-64b6f9445d-zr8j9      1/1     Running   0          4m53s   10.244.0.5   minikube   <none>           <none>
pod/heart-api-7cddd6db4f-44tq2    1/1     Running   0          4m41s   10.244.0.8   minikube   <none>           <none>
pod/heart-api-7cddd6db4f-xcnz8    1/1     Running   0          4m52s   10.244.0.7   minikube   <none>           <none>
pod/prometheus-5f5c5f4c4-zthtq    1/1     Running   0          4m53s   10.244.0.6   minikube   <none>           <none>

NAME                 TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE     SELECTOR
service/grafana      LoadBalancer   10.106.169.109   <pending>     3000:31957/TCP   4m53s   app=grafana
service/heart-api    LoadBalancer   10.103.234.102   <pending>     8000:31694/TCP   12m     app=heart-api
service/kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP          13m     <none>
service/prometheus   LoadBalancer   10.101.56.36     <pending>     9090:30275/TCP   4m53s   app=prometheus
```

*Figure 15. kubectl get pods,svc on the cluster used for the screenshots.*

The form opened at minikube service heart-api --url posts to /predict inside the cluster. Pod logs show the same line the process logger writes, for example POST /predict -> 200 in 13.2ms (kubectl logs -l app=heart-api --tail=50).

# 11. Monitoring and logging

Every API request except /metrics is recorded by RequestMonitor (src/mlops/monitor.py):

- A log line with method, path, status code and latency in ms, visible through kubectl logs.
- The last 100 events kept in memory and rendered on /monitor.
- Two Prometheus series: counter heart_api_requests_total (method, path, status) and histogram heart_api_request_latency_seconds (method, path).

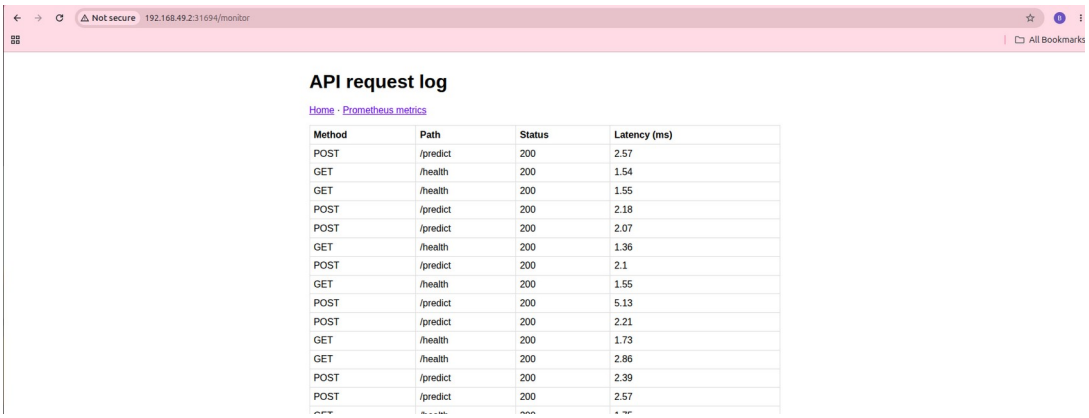



*Figure 16. In-memory request log on /monitor.*

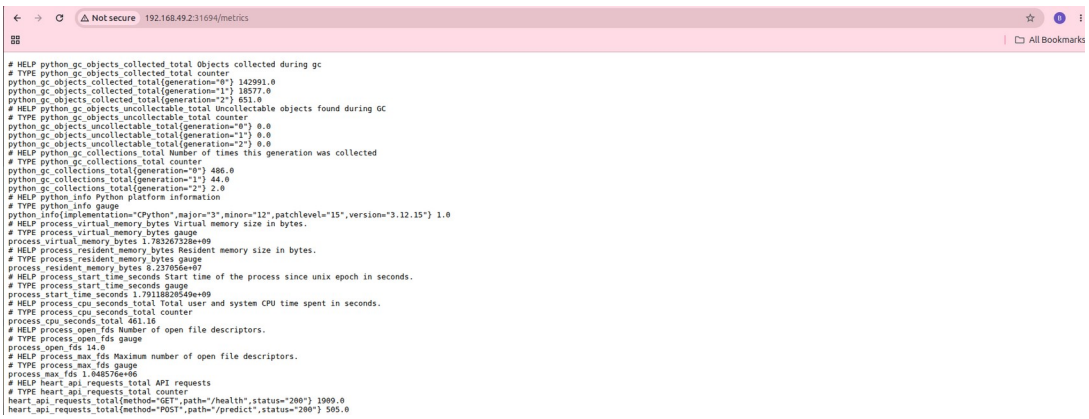



*Figure 17. Raw /metrics output.*

Prometheus pulls metrics; the app does not push them. k8s/prometheus.yaml scrapes heart-api:8000/metrics every 5 seconds (Prometheus 3.2.1). On Status > Targets the job heart-api is **1/1 up**, last scrape about 8 ms.

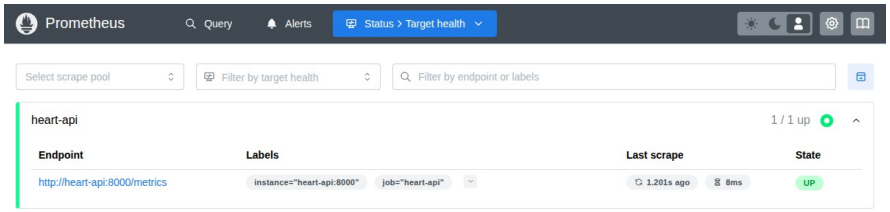



*Figure 18. Prometheus target health.*

Grafana queries Prometheus rather than the API. k8s/grafana.yaml (Grafana 11.6.0) provisions the **Prometheus** datasource at http://prometheus:9090 and the dashboard **Heart Disease API** with two panels: request rate sum(rate(heart_api_requests_total[1m])) by path and status, and p95 latency from heart_api_request_latency_seconds_bucket. Login is admin / admin with sign-up off.

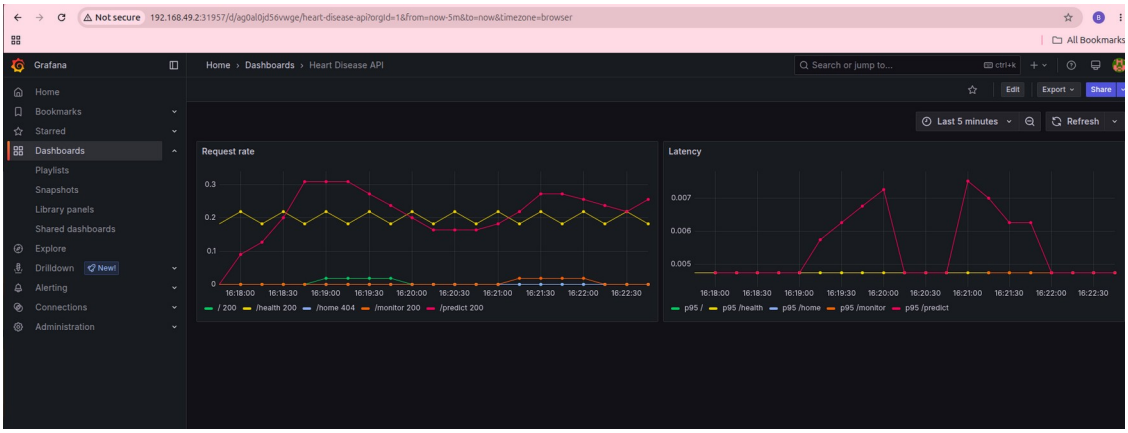



*Figure 19. Grafana dashboard: Heart Disease API.*

# 12. Reproducing the deployment

From a clean checkout with Docker, Minikube and kubectl installed:

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
PYTHONPATH=src python -m mlops.main
PYTHONPATH=src pytest tests/ -q
docker build -t heart-api:local .
minikube start --driver=docker
minikube image load heart-api:local
kubectl apply -f k8s/
minikube service heart-api --url
minikube service prometheus --url
minikube service grafana --url
```

No public-cloud URL is used; access for marking is the Minikube service URL above plus the repository. Stop the cluster with `minikube stop` and restart it with `minikube start --driver=docker`; reload the image only after a rebuild.

# 13. Limitations and future work

- **Small dataset.** 297 rows and a 60-row holdout make test metrics noisy; the 0.01 AUC gap between the two models is within that noise. Repeated cross-validation would give a firmer comparison.
- **Recall.** About 0.72 recall misses some disease cases. A lower threshold or a recall-weighted objective would suit a screening use case.
- **Model registry and promotion.** The ONNX file is copied into the image at build time. A registry-based promotion step, with CI building and pushing the image, would complete the CD half of the pipeline.
- **Drift monitoring.** Monitoring covers latency and request counts, not input or prediction drift; adding feature-distribution metrics and alerts is the next step.
- **Deployment target.** Minikube is a local cluster; a managed cluster with ingress, TLS, autoscaling and a Helm chart would be the production version.